



Universidad Autónoma del Estado de México

Centro Universitario UAEM Atlacomulco



Licenciatura en Ingeniería en Computación

Periodo educativo: **2026-B**

Grupo: ICO-30

A04 Ejercicios de polimorfismo UML y Código

Paradigmas de la programación I

Alumno:

Emanuel García Banderas

Docente: Julio Alberto De La Teja López

Atlacomulco, Estado de México, a 2 de septiembre del 2026

1. Introduction

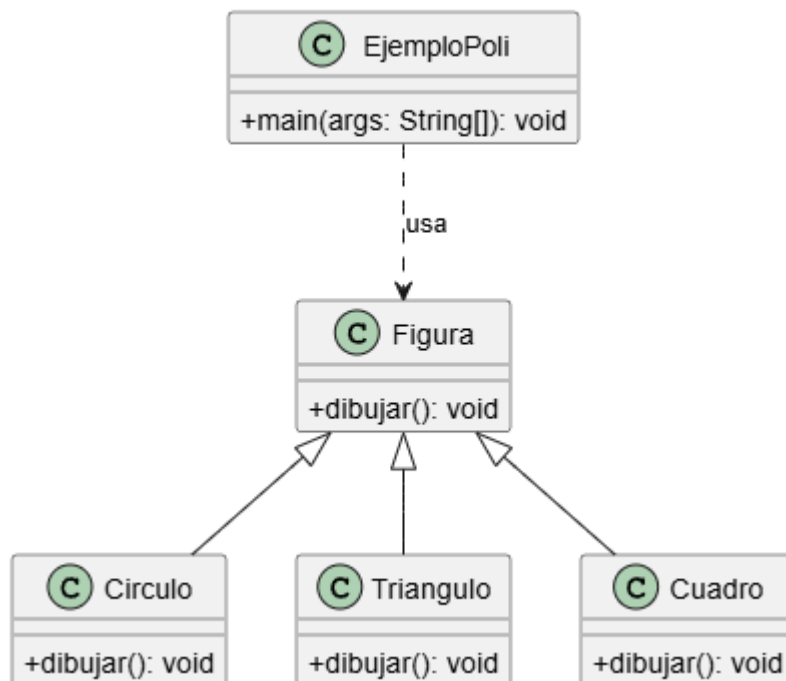
This report focuses on the implementation of polymorphism in Object-Oriented Programming using Java. Polymorphism allows a single interface or reference type to represent different underlying object types. Through these exercises, we explore both runtime polymorphism (method overriding via class hierarchies) and compile-time polymorphism (method overloading). The objective is to demonstrate how these principles create flexible, scalable, and highly cohesive systems.

2. Development

Exercise 1: Runtime Polymorphism with Shapes

Why this is polymorphism: This exercise demonstrates runtime polymorphism by using a single array of the base type *Figura* to hold different child objects (*Circulo*, *Triangulo*, *Cuadro*). When the *dibujar()* method is called inside a generic loop, Java dynamically determines at runtime which specific implementation to execute based on the actual object instantiated in memory, rather than the reference type of the array.

- **UML Diagram:**



- **Source Code:**

```
public class EjemploPoli {
```

```

    public static void main(String[] args) {
        Triangulo fig = new Triangulo();
        Cuadro fig2 = new Cuadro();
        Circulo fig3 = new Circulo();

        fig.dibujar();
        fig2.dibujar();
        fig3.dibujar();

        Figura[] figus = new Figura[3];
        figus[0] = new Circulo();
        figus[1] = new Triangulo();
        figus[2] = new Cuadro();

        for (Figura f : figus) {
            f.dibujar();
        }
    }
}

class Figura {
    public void dibujar() {
        System.out.println("Dibujando una figura");
    }
}

class Circulo extends Figura {
    @Override
    public void dibujar() {
        System.out.println("Dibujando un circulo");
    }
}

class Triangulo extends Figura {
    public void dibujar() {
        System.out.println("Dibujando un triangulo");
    }
}

class Cuadro extends Figura {
    @Override
    public void dibujar() {
        System.out.println("dibujando un cuadrado");
    }
}
}

```

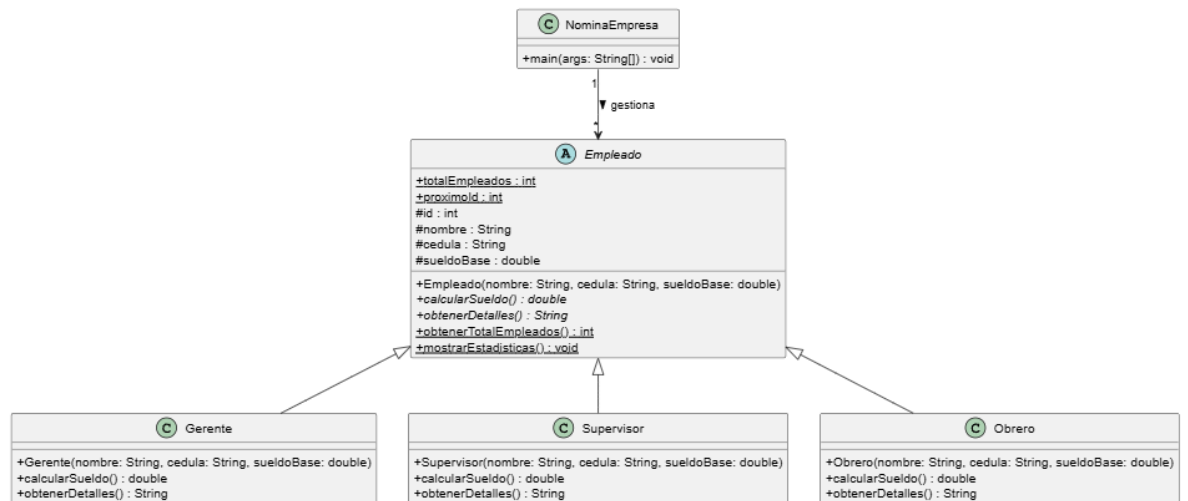
OUT:

```
Dibujando un triangulo
dibujando un cuadrado
Dibujando un circulo
Dibujando un circulo
Dibujando un triangulo
dibujando un cuadrado
PS C:\Users\emanu\OneDrive\Documentos\paradigmas>
```

Exercise 2: Polymorphic Payroll System

Why this is polymorphism: The payroll system leverages polymorphism by interacting with a generic Empleado array that contains specific Gerente, Supervisor, and Obrero objects. Even though the loop iterates over abstract Empleado references, calling calcularSueldo() and obtenerDetalles() triggers the unique logic overridden in each specific subclass, allowing the system to process all employees uniformly without needing multiple if/else statements to check their roles.

- **UML Diagram:** [Insert screenshot of Empleados.puml here]



- **Source Code:**

```
abstract class Empleado {
    protected static int totalEmpleados = 0;
    protected static int proximoId = 1001;

    protected int id;
    protected String nombre;
    protected String cedula;
    protected double sueldoBase;
```

```

    public Empleado(String nombre, String cedula, double sueldoBase) {
        this.id = proximoId++;
        this.nombre = nombre;
        this.cedula = cedula;
        this.sueldoBase = sueldoBase;
        totalEmpleados++;
    }

    public abstract double calcularSueldo();
    public abstract String obtenerDetalles();

    public static int obtenerTotalEmpleados() {
        return totalEmpleados;
    }

    public static void mostrarEstadisticas() {
        System.out.println("Total empleados: " + totalEmpleados);
    }
}

class Gerente extends Empleado {
    public Gerente(String nombre, String cedula, double sueldoBase) {
        super(nombre, cedula, sueldoBase);
    }

    @Override
    public double calcularSueldo() {
        return sueldoBase * 1.25;
    }

    @Override
    public String obtenerDetalles() {
        return String.format("[%d] %s - Gerente - $%.2f", id, nombre,
calcularSueldo());
    }
}

class Supervisor extends Empleado {
    public Supervisor(String nombre, String cedula, double sueldoBase) {
        super(nombre, cedula, sueldoBase);
    }

    @Override
    public double calcularSueldo() {
        return sueldoBase * 1.15;
    }
}

```

```

    }

    @Override
    public String obtenerDetalles() {
        return String.format("[%d] %s - Supervisor - $%.2f", id, nombre,
calcularSueldo());
    }
}

class Obrero extends Empleado {
    public Obrero(String nombre, String cedula, double sueldoBase) {
        super(nombre, cedula, sueldoBase);
    }

    @Override
    public double calcularSueldo() {
        return sueldoBase;
    }

    @Override
    public String obtenerDetalles() {
        return String.format("[%d] %s - Obrero - $%.2f", id, nombre,
calcularSueldo());
    }
}

public class NominaEmpresa {
    public static void main(String[] args) {
        Empleado[] empleados = new Empleado[8];

        empleados[0] = new Gerente("Laura", "CED-01", 4000.00);
        empleados[1] = new Gerente("Roberto", "CED-02", 4500.00);
        empleados[2] = new Supervisor("Ana", "CED-03", 2500.00);
        empleados[3] = new Supervisor("Pedro", "CED-04", 2700.00);
        empleados[4] = new Obrero("Carlos", "CED-05", 1500.00);
        empleados[5] = new Obrero("María", "CED-06", 1600.00);
        empleados[6] = new Obrero("Juan", "CED-07", 1550.00);
        empleados[7] = new Obrero("Sofía", "CED-08", 1580.00);

        double totalNomina = 0;

        for (Empleado emp : empleados) {
            System.out.println(emp.obtenerDetalles());
            totalNomina += emp.calcularSueldo();
        }
    }
}

```

```

        System.out.printf("Total: $%.2f%n", totalNomina);
        Empleado.mostrarEstadisticas();
    }
}

```

OUT:

```

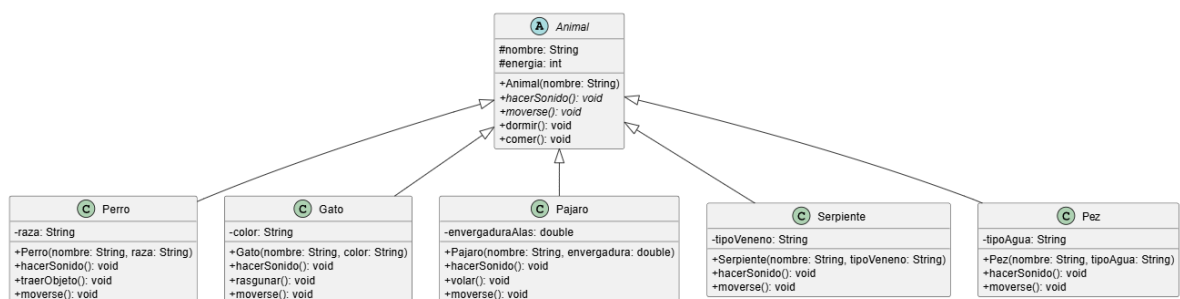
[1001] Laura - Gerente - $5000.00
[1002] Roberto - Gerente - $5625.00
[1003] Ana - Supervisor - $2875.00
[1004] Pedro - Supervisor - $3105.00
[1005] Carlos - Obrero - $1500.00
[1006] María - Obrero - $1600.00
[1007] Juan - Obrero - $1550.00
[1008] Sofía - Obrero - $1580.00
Total: $22835.00
Total empleados: 8
PS C:\Users\emanu\OneDrive\Documentos\paradigmas>

```

Exercise 3: Animal Shelter Polymorphic Collection

Why this is polymorphism: This exhibits polymorphism through an array of the abstract Animal class. Calling hacerSonido() and moverse() on each element invokes the distinct, overridden behaviors of the underlying Perro, Gato, Pajaro, Serpiente, and Pez objects. The program can process an entire shelter of animals generically without knowing the specific species of each element in advance.

- **UML Diagram:**



- **Source Code:**

```

abstract class Animal {
    protected String nombre;
    protected int energia;

    public Animal(String nombre) {

```

```

        this.nombre = nombre;
        this.energia = 100;
    }

    public abstract void hacerSonido();
    public abstract void moverse();

    public void dormir() {
        System.out.println(nombre + " zzzzzzz");
        this.energia += 20;
    }

    public void comer() {
        System.out.println(nombre + " está comiendo");
        this.energia += 10;
    }
}

class Perro extends Animal {
    private String raza;

    public Perro(String nombre, String raza) {
        super(nombre);
        this.raza = raza;
    }

    @Override
    public void hacerSonido() {
        System.out.println(nombre + " ladra: Guau guau");
    }

    @Override
    public void moverse() {
        System.out.println(nombre + " corre ");
    }

    public void traerObjeto() {
        System.out.println(nombre + " trajo el objeto");
    }
}

class Gato extends Animal {
    private String color;

    public Gato(String nombre, String color) {

```



```

        super(nombre);
        this.color = color;
    }

    @Override
    public void hacerSonido() {
        System.out.println(nombre + " maúlla: Miau miau");
    }

    @Override
    public void moverse() {
        System.out.println(nombre + " camina sigilosamente");
    }

    public void rasgunar() {
        System.out.println(nombre + " está rasguñando el sillón");
    }
}

class Pajaro extends Animal {
    private double envergaduraAlas;

    public Pajaro(String nombre, double envergaduraAlas) {
        super(nombre);
        this.envergaduraAlas = envergaduraAlas;
    }

    @Override
    public void hacerSonido() {
        System.out.println(nombre + " canta: Pío pío");
    }

    @Override
    public void moverse() {
        System.out.println(nombre + " vuela por el cielo");
    }

    public void volar() {
        System.out.println(nombre + " está volando alto");
    }
}

class Serpiente extends Animal {
    private String tipoVeneno;

```

```

    public Serpiente(String nombre, String tipoVeneno) {
        super(nombre);
        this.tipoVeneno = tipoVeneno;
    }

    @Override
    public void hacerSonido() {
        System.out.println(nombre + " sisea: ssss ssss");
    }

    @Override
    public void moverse() {
        System.out.println(nombre + " se arrastra por el suelo");
    }
}

class Pez extends Animal {
    private String tipoAgua;

    public Pez(String nombre, String tipoAgua) {
        super(nombre);
        this.tipoAgua = tipoAgua;
    }

    @Override
    public void hacerSonido() {
        System.out.println(nombre + " hace burbujas: Glu glu");
    }

    @Override
    public void moverse() {
        System.out.println(nombre + " nada velozmente en el agua");
    }
}

public class aaa {
    public static void main(String[] args) {
        Animal[] refugio = new Animal[5];
        refugio[0] = new Perro("Max", "Golden Retriever");
        refugio[1] = new Gato("Luna", "Negro");
        refugio[2] = new Pajaro("Piolín", 25.5);
        refugio[3] = new Serpiente("Kaa", "Neurotóxico");
        refugio[4] = new Pez("Nemo", "Agua Dulce");

        for (Animal animal : refugio) {

```

```

        System.out.println("\nAnimal: " + animal.nombre);
        animal.hacerSonido();
        animal.moverse();
    }
}
}

```

OUT:

```

Animal: Max
Max ladra: Guau guau
Max corre

Animal: Luna
Luna maúlla: Miau miau
Luna camina sigilosamente

Animal: Piolín
Piolín canta: Pío pío
Piolín vuela por el cielo

Animal: Kaa
Kaa sisea: ssss ssss
Kaa se arrastra por el suelo

Animal: Nemo
Nemo hace burbujas: Glu glu
Nemo nada velozmente en el agua
PS C:\Users\emanu\OneDrive\Documentos\paradigmas>

```

3.Conclusion

The presented exercises thoroughly demonstrate the practical applications of polymorphism. Whether using method overloading to adapt functions to different data types or method overriding to give distinct behaviors to subclasses in an array, polymorphism proves essential for modular programming. It enables developers to write generic code that can seamlessly interface with a wide array of specific objects, simplifying architecture and significantly reducing code complexity.

Nombre del docente: Julio Alberto De La Teja López				
Nombre del estudiante: Emanuel García Banderas				
Indicador	Nivel de desempeño			Valor asignado
	Alto (.5)	Medio (.3)	Bajo (.1)	
Funcionalidad	La solución es completamente funcional y produce resultados precisos en todos los casos.	La solución produce resultados correctos en la mayoría de los casos, con algunos errores menores.	La solución tiene errores importantes y no produce los resultados esperados.	0.5
Claridad	El código es claro, con comentarios detallados que enriquecen la comprensión.	El código está comentado y utiliza nombres descriptivos, lo que facilita la comprensión.	El código es confuso y difícil de entender debido a la falta de comentarios y nombres descriptivos.	0.5
Creatividad	La solución es creativa, innovadora y muestra una perspectiva única.	La solución muestra un intento de creatividad, pero no es totalmente original.	La solución es una copia directa de ejemplos previos o carece de enfoque creativo.	0.5
Entrega de reporte	El reporte se entrega con estructura, Introducción, desarrollo (código), capturas de salida y ejemplos de funcionamiento, comentarios.	El reporte solo contiene tres de los cuatro apartados elementos.	El reporte no tiene la estructura especificada.	0.5

TOTAL= 2.0